

SILESIAIAN UNIVERSITY OF TECHNOLOGY

Internet Banking System

Technical Project Report

Piotr Krupiński

Jeremi Szczotka

Szymon Mańka

Hyunseok Cho

Simão Galvão Rosado Silva Pereira

June 10, 2026

Contents

1	Introduction	6
1.1	Project Motivation and Goals	6
1.2	Academic Context	6
1.3	Scope and Constraints	7
1.4	Document Structure	7
2	System Overview	8
2.1	Three-Tier Architecture	8
2.2	Technology Stack	9
2.3	Docker Compose Orchestration	9
3	Requirements	10
3.1	User Roles	10
3.2	Functional Requirements	10
3.2.1	Authentication	10
3.2.2	Customer Operations	10
3.2.3	Admin Operations	11
3.3	Non-Functional Requirements	11
4	Database Design	12
4.1	Design Process	12
4.2	ERD Tooling	12
4.3	Schema Overview	13
4.4	Core Entity Tables	14
4.4.1	users	14
4.4.2	bank_accounts	14
4.4.3	transactions	15
4.4.4	block_requests	15
4.4.5	otp_sessions	15
4.4.6	operation_records	16
4.5	Lookup Tables	16
4.6	Entity Relationships	16
4.7	Key Design Decisions	17
5	Backend Architecture	18
5.1	Package Structure	18
5.2	Security Filter Chain	18
5.3	JWT Implementation	19

5.4	Two-Factor Authentication Flow	19
5.5	Service Layer	20
5.6	Exception Handling	20
6	Frontend Architecture	21
6.1	Overview	21
6.2	Routing	21
6.3	Authentication Context	21
6.4	API Client	22
7	API Design	23
7.1	Endpoint Overview	23
7.2	HTTP Status Codes	24
8	Security Model	25
8.1	Two-Factor Authentication	25
8.2	Account Locking	25
8.3	Account Blocking Workflow	25
8.4	Audit Logging	26
8.5	Known Limitations	26
9	Testing	27
9.1	Test Strategy	27
9.2	Unit Tests	27
9.3	Integration Tests	28
9.4	Test Results	28
9.5	Coverage Gaps	28
10	Application Walkthrough	29
10.1	Landing Page	29
10.2	Login and Two-Factor Authentication	30
10.3	Admin Dashboard	31
10.4	Customer Dashboard	34
11	Deployment	36
11.1	Docker Compose Setup	36
11.2	Environment Variables	36
11.3	Starting the Application	36
12	Conclusions	38
12.1	Summary	38
12.2	Lessons Learned	38
12.3	Potential Improvements	39
	References	40

List of Figures

2.1	Admin dashboard showing system-wide statistics — evidence that all three tiers are communicating correctly.	8
4.1	Entity-Relationship Diagram — full schema with 16 tables and all foreign key relationships.	13
10.1	Landing page — hero section. The public homepage presents the platform concept and provides navigation to login and register.	29
10.2	Landing page — roles section. Describes the two primary roles (Customer and Administrator) and offers quick access to the login modal with pre-filled demo credentials.	30
10.3	Login modal — step 1. The user enters their email and password. Demo credential shortcuts are provided for each role.	30
10.4	OTP verification modal — step 2. After a successful password check, the user must enter the 6-digit OTP code retrieved from the backend log ([DEV] OTP for admin@bank.local: 022006).	31
10.5	Admin overview — system statistics. Shows total customers (3), total funds across all accounts (26,855.58), blocked users (1), and pending block requests (1).	31
10.6	Admin overview — recent high-priority events. The dashboard shows the latest CRITICAL and WARNING events from the audit log, including the locked customer incident and Brian’s block request.	32
10.7	Admin security queue. Three sections: pending block requests (Brian Walsh’s request with Approve/Reject buttons), blocked access users (Locked Customer with Restore access), and blocked accounts (empty at this point).	32
10.8	Admin operations — full audit log. Every action is recorded: account unblock, account block, access restore, login success, login failure, block request, and payment. Searchable and filterable by severity and type. .	33
10.9	Admin customer administration. Lists all customers with access status, failed attempts, account counts, pending requests, and last login. A “Manage” action opens a customer detail drawer.	33
10.10	Customer overview — Alice Murphy’s dashboard. Shows total balance (24,665.18), 2 active accounts, and the 5 most recent transactions with types, statuses, and amounts.	34
10.11	Customer accounts page. Shows both accounts with IBAN, account number, currency, balance, and status. The detail panel provides download and block request actions.	34

10.12Payments and transfers page. The Transfer tab allows sending funds to an account number. The Bill Payment tab handles external payees. Only ACTIVE accounts are shown in the source account selector. . . . 35

10.13Transaction history page. Shows all transactions with date, account, type, title, counterparty, status, and signed amount. Filterable by account and transaction type, with a CSV download option. 35

List of Tables

2.1	Technology stack	9
3.1	User roles and their primary capabilities	10
3.2	Non-functional requirements	11
4.1	<code>users</code> table	14
4.2	<code>bank_accounts</code> table	14
4.3	<code>transactions</code> table	15
4.4	<code>block_requests</code> table	15
4.5	<code>otp_sessions</code> table	15
4.6	<code>operation_records</code> table (audit log)	16
4.7	Lookup table structure (applied to all 10 domain tables)	16
4.8	Lookup tables and their code values	16
4.9	Entity relationships	17
5.1	Core service classes and their responsibilities	20
5.2	Exception to HTTP status mapping	20
7.1	REST API endpoints	23
7.2	HTTP status codes used by the API	24
8.1	Audit log severity levels and example events	26
8.2	Known security limitations	26
9.1	Unit test classes and scenario counts	27
9.2	Integration test classes	28
9.3	Known test coverage gaps	28
11.1	Docker Compose services	36
11.2	Key environment variables	36

Chapter 1

Introduction

1.1 Project Motivation and Goals

The Internet Banking System is a full-stack web application developed as a university project to demonstrate the design and implementation of a secure, role-based digital banking platform. The system is intended to simulate the core operations of a modern retail bank's online portal, covering customer-facing features such as account management, fund transfers, and transaction history, as well as administrative capabilities for managing customers, monitoring security events, and handling account blocking workflows.

The primary goal of the project was to build a production-quality prototype that showcases how security-critical applications are engineered: how authentication is layered (password plus one-time password), how access control is enforced at the API level, how financial data is modelled in a relational database, and how every security-relevant action is recorded in an immutable audit log.

A secondary goal was to apply academic database design principles in practice. The database schema was designed iteratively — first using Java enums stored as plain strings, and then refactored to use the *enum-as-table* (lookup table) pattern, which enforces referential integrity at the database level and allows domain values to be queried, labelled, and extended without modifying application code.

1.2 Academic Context

This project was completed as part of a database systems course. The requirements included:

- Design a relational database schema with proper normalisation.
- Use lookup tables (reference data tables) for enumerated values.
- Draw and document an Entity-Relationship Diagram (ERD).
- Implement a working application backed by the designed schema.
- Demonstrate the system with real end-to-end scenarios.

1.3 Scope and Constraints

The system implements the following scope:

- Three user roles: Customer, Admin, and Employee (placeholder).
- Two-factor authentication (2FA) using a one-time password (OTP).
- Customer operations: view accounts, transfer funds, pay bills, request account block, download statements.
- Admin operations: dashboard statistics, customer management, security queue (approve/reject block requests, unlock accounts), full audit log.
- Containerised deployment using Docker Compose.

Out of scope for this prototype: email/SMS delivery for OTP (codes are logged to the console), real payment gateway integration, and multi-currency conversion.

1.4 Document Structure

The remainder of this document is organised as follows. Chapter 2 describes the high-level system architecture. Chapter 3 lists the functional and non-functional requirements. Chapter 4 covers the database design process in detail, including the ERD and schema decisions. Chapter 5 describes the backend architecture and security mechanisms. Chapter 6 covers the frontend. Chapter 7 documents the REST API. Chapter 8 analyses the security model. Chapter 9 describes the testing strategy and results. Chapter 10 provides a visual walkthrough of the application. Chapter 11 explains the deployment setup. Chapter 12 concludes with lessons learned and future improvements.

Chapter 2

System Overview

2.1 Three-Tier Architecture

The system follows a classic three-tier client-server architecture:

1. **Presentation tier** — a React single-page application (SPA) served by Nginx, communicating with the backend via a JSON REST API.
2. **Application tier** — a Spring Boot REST API running on port 8080, implementing all business logic, authentication, and authorisation.
3. **Data tier** — a PostgreSQL 16 relational database running on port 5432, storing all persistent data.

All three tiers are containerised and orchestrated with Docker Compose.

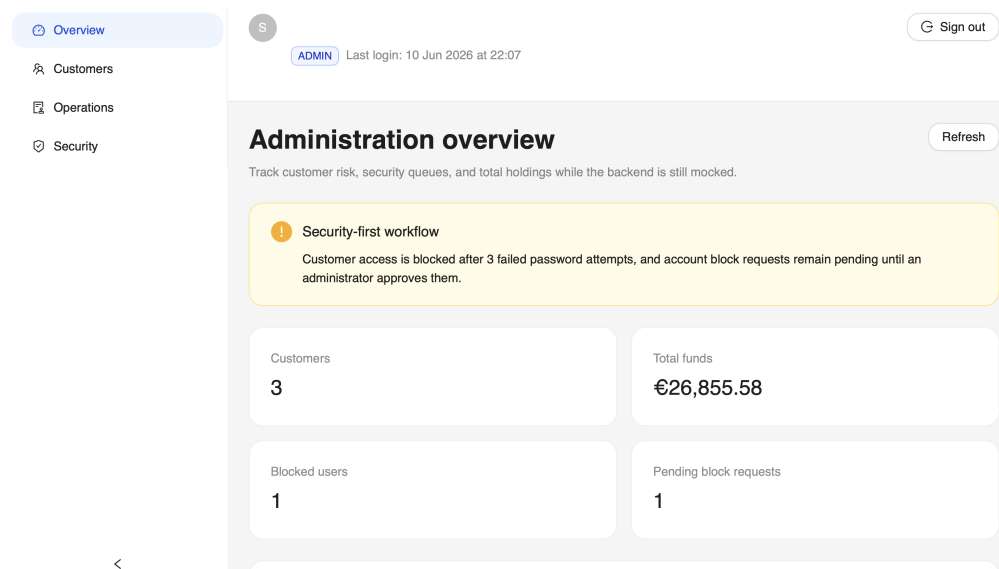


Figure 2.1: Admin dashboard showing system-wide statistics — evidence that all three tiers are communicating correctly.

2.2 Technology Stack

Table 2.1 summarises the technologies used in each tier.

Table 2.1: Technology stack

Tier	Technology	Version
Frontend	React	18.3.1
	TypeScript	5.6.3
	Vite (build tool)	5.4.10
	Ant Design (UI)	5.27.1
	React Router	6.28
Backend	Java	21
	Spring Boot	3.3.5
	Spring Security	6.x
	Spring Data JPA	3.3.5
	Hibernate	6.5.3
	JWT (JWT library)	0.12.6
	Lombok	latest
Database	PostgreSQL	16
	H2 (tests only)	—
DevOps	Docker	24+
	Docker Compose	v2
	Maven	3.9.8
Testing	JUnit 5	—
	Mockito	—
	Playwright	1.60.0

2.3 Docker Compose Orchestration

Three services are defined in `docker-compose.yml`: `ib-db` (PostgreSQL), `ib-backend` (Spring Boot), and `ib-frontend` (Nginx). The startup order is enforced by `depends_on`: the database must be healthy before the backend starts, and the backend must be running before the frontend starts.

Starting the application

```
docker compose up --build
```

The backend is built inside Docker using a multi-stage Maven build, so no local Java installation is required. The frontend uses a multi-stage Node.js build.

Chapter 3

Requirements

3.1 User Roles

Table 3.1: User roles and their primary capabilities

Role	Capabilities
CUSTOMER	Register, log in (2FA), view accounts and balances, make transfers, make payments, view transaction history, download CSV statements, request account block.
ADMIN	All customer-visible data, manage customers, approve/reject block requests, block/unblock accounts directly, restore locked user access, view full audit log, view system-wide statistics.
EMPLOYEE	Reserved for future expansion. Currently a placeholder.

3.2 Functional Requirements

3.2.1 Authentication

- FR-01: Users shall authenticate using email and password.
- FR-02: A one-time password (OTP) shall be required after successful password verification (two-factor authentication).
- FR-03: After three consecutive incorrect passwords, the account shall be locked automatically.
- FR-04: Locked accounts shall be unlocked only by an administrator.
- FR-05: OTP sessions shall expire after five minutes.

3.2.2 Customer Operations

- FR-10: Customers shall be able to view all their bank accounts with balances.

- FR-11: Customers shall be able to transfer funds to another account by account number.
- FR-12: If the recipient account number belongs to the same system, both sides of the transfer shall be recorded (debit and credit transactions).
- FR-13: Customers shall be able to pay bills to external payees.
- FR-14: Customers shall be able to download a CSV account statement.
- FR-15: Customers shall be able to request that one of their accounts be blocked, providing a reason.

3.2.3 Admin Operations

- FR-20: Administrators shall see a dashboard with total customers, total funds, blocked users, pending block requests, and recent high-severity events.
- FR-21: Administrators shall be able to approve or reject customer block requests.
- FR-22: Administrators shall be able to block or unblock accounts directly.
- FR-23: Administrators shall be able to restore access for locked customers.
- FR-24: Administrators shall be able to view the complete audit log.

3.3 Non-Functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Security	Passwords stored as BCrypt hashes. JWT tokens signed with HMAC-SHA256. All sensitive actions recorded in audit log.
Statelessness	API is fully stateless — no server-side sessions.
Auditability	Every security-relevant action produces an <code>operation_record</code> entry with actor, target, severity, and description.
Data Integrity	Monetary values use <code>DECIMAL(19,2)</code> to avoid floating-point errors. All foreign keys are enforced at the database level.
Containerisation	Full system runs with a single <code>docker compose up --build</code> command.
Testability	Unit and integration tests cover all core business flows. 46 tests pass on every build.

Chapter 4

Database Design

4.1 Design Process

The database schema went through two major iterations during the project.

Iteration 1 — Enums as VARCHAR strings. In the first iteration, all enumerated values (roles, statuses, types) were stored directly as `VARCHAR` columns using the JPA `@Enumerated(EnumType.STRING)` annotation. This approach is convenient during rapid development but has a drawback: the database has no knowledge of which values are valid. A typo or an invalid string inserted directly into the database would not be caught by any constraint.

Iteration 2 — Lookup tables (enum-as-table pattern). Following the course requirement and best practices in relational database design, the schema was refactored to use separate lookup tables for every enumerated domain. Each lookup table has three columns: `id` (UUID primary key), `code` (a unique short string matching the Java enum name, e.g. "ACTIVE"), and `label` (a human-readable description). The main entity tables reference these lookup tables via foreign keys instead of storing the string directly.

The advantages of this approach are:

- **Referential integrity enforced by the database:** an invalid value cannot be inserted because the foreign key constraint will reject it.
- **Self-documenting schema:** all valid values are visible as rows in the lookup table.
- **Extensible without a code deploy:** a new status value can be added by inserting a row into the lookup table, without changing the application code or redeploying.
- **Standard relational design:** follows 3rd normal form.

4.2 ERD Tooling

The ERD was designed using dbdiagram.io, a browser-based tool that accepts a DBML (Database Markup Language) description and renders an interactive diagram.

The SQL DDL was also maintained in `docs/erd.sql` to keep the schema under version control.

Figure 4.1 shows the final ERD exported from dbdiagram.io.

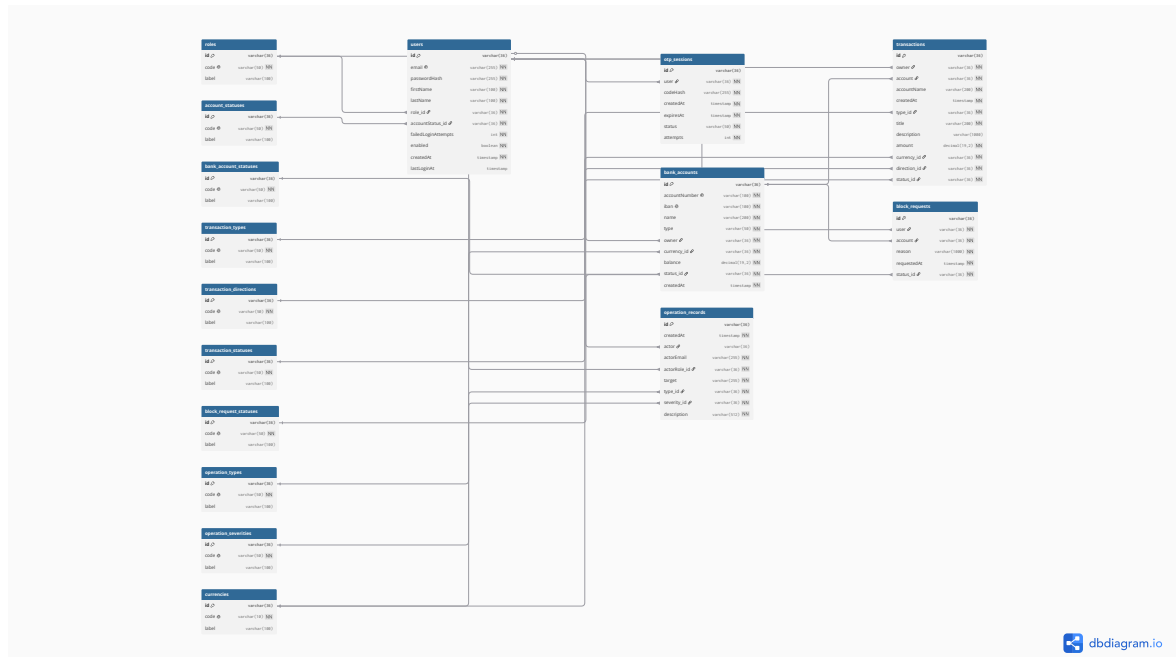


Figure 4.1: Entity-Relationship Diagram — full schema with 16 tables and all foreign key relationships.

4.3 Schema Overview

The schema consists of **16 tables** in three groups:

- **6 core entity tables:** users, bank_accounts, transactions, block_requests, otp_sessions, operation_records.
- **10 lookup tables:** roles, account_statuses, bank_account_statuses, transaction_types, transaction_directions, transaction_statuses, block_request_statuses, operation_types, operation_severities, currencies.

All primary keys are UUIDs generated by the application (not database sequences), which avoids exposing sequential identifiers to clients and makes distributed generation possible in future.

4.4 Core Entity Tables

4.4.1 users

Table 4.1: users table

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
email	varchar(255)	UNIQUE, NOT NULL	Login identifier
passwordHash	varchar(255)	NOT NULL	BCrypt hash
firstName	varchar(100)	NOT NULL	
lastName	varchar(100)	NOT NULL	
role_id	varchar(36)	FK → roles	
accountStatus_id	varchar(36)	FK → account_statuses	
failedLoginAttempts	int	NOT NULL, default 0	
enabled	boolean	NOT NULL	Spring Security flag
createdAt	timestamp	NOT NULL	
lastLoginAt	timestamp	nullable	Updated on OTP verify

4.4.2 bank_accounts

Table 4.2: bank_accounts table

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
accountNumber	varchar(100)	UNIQUE, NOT NULL	Internal number
iban	varchar(100)	UNIQUE, NOT NULL	IBAN format
name	varchar(200)	NOT NULL	Display name
type	varchar(50)	NOT NULL	e.g. Current, Savings
owner	varchar(36)	FK → users	
currency_id	varchar(36)	FK → currencies	
balance	decimal(19,2)	NOT NULL, default 0	
status_id	varchar(36)	FK → bank_account_statuses	
createdAt	timestamp	NOT NULL	

4.4.3 transactions

Table 4.3: transactions table

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
owner	varchar(36)	FK → users	Index: idx_tx_owner
account	varchar(36)	FK → bank_accounts	Index: idx_tx_account
accountName	varchar(200)	NOT NULL	Snapshot at time of TX
createdAt	timestamp	NOT NULL	Index: idx_tx_created
type_id	varchar(36)	FK → transaction_types	
title	varchar(200)	NOT NULL	
description	varchar(1000)	nullable	
amount	decimal(19,2)	NOT NULL	Always positive
currency_id	varchar(36)	FK → currencies	
direction_id	varchar(36)	FK → transaction_directions	DEBIT or CREDIT
status_id	varchar(36)	FK → transaction_statuses	

4.4.4 block_requests

Table 4.4: block_requests table

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
user	varchar(36)	FK → users	Who submitted the request
account	varchar(36)	FK → bank_accounts	Which account to block
reason	varchar(1000)	NOT NULL	Customer's explanation
requestedAt	timestamp	NOT NULL	
status_id	varchar(36)	FK → block_request_statuses	

4.4.5 otp_sessions

Table 4.5: otp_sessions table

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
user	varchar(36)	FK → users	
codeHash	varchar(255)	NOT NULL	BCrypt hash of OTP code
createdAt	timestamp	NOT NULL	
expiresAt	timestamp	NOT NULL	now + 5 minutes
status	varchar(50)	NOT NULL	PENDING / USED / EXPIRED
attempts	int	NOT NULL, default 0	Blocks after 3 wrong tries

4.4.6 operation_records

Table 4.6: operation_records table (audit log)

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
createdAt	timestamp	NOT NULL	Index
actor	varchar(36)	FK → users, nullable	Null for system events
actorEmail	varchar(255)	NOT NULL	Denormalised snapshot
actorRole_id	varchar(36)	FK → roles	Index
target	varchar(255)	NOT NULL	Index
type_id	varchar(36)	FK → operation_types	
severity_id	varchar(36)	FK → operation_severities	
description	varchar(512)	NOT NULL	

4.5 Lookup Tables

All ten lookup tables follow the same structure:

Table 4.7: Lookup table structure (applied to all 10 domain tables)

Column	Type	Constraints	Notes
id	varchar(36)	PK	UUID
code	varchar(50)	UNIQUE, NOT NULL	Matches Java enum name
label	varchar(100)	nullable	Human-readable description

Table 4.8 lists all ten lookup tables and their valid code values.

Table 4.8: Lookup tables and their code values

Table	Valid code values
roles	ADMIN, EMPLOYEE, CUSTOMER
account_statuses	ACTIVE, LOCKED_LOGIN_FAILURE, BLOCKED_BY_BANK, BL
bank_account_statuses	ACTIVE, PENDING_BLOCK, BLOCKED
transaction_types	TRANSFER, PAYMENT, DEPOSIT
transaction_directions	DEBIT, CREDIT
transaction_statuses	COMPLETED, PENDING, FAILED
block_request_statuses	PENDING, APPROVED, REJECTED
operation_types	LOGIN_FAILURE, LOGIN_SUCCESS, OTP_VERIFIED, TRANSFE
operation_severities	INFO, WARNING, CRITICAL, SUCCESS
currencies	EUR (and others as needed)

4.6 Entity Relationships

Table 4.9 lists all foreign key relationships in the schema.

Table 4.9: Entity relationships

From	To	Type
users	roles	Many-to-One
users	account_statuses	Many-to-One
bank_accounts	users	Many-to-One (owner)
bank_accounts	bank_account_statuses	Many-to-One
bank_accounts	currencies	Many-to-One
transactions	users	Many-to-One (owner)
transactions	bank_accounts	Many-to-One
transactions	transaction_types	Many-to-One
transactions	transaction_directions	Many-to-One
transactions	transaction_statuses	Many-to-One
transactions	currencies	Many-to-One
block_requests	users	Many-to-One
block_requests	bank_accounts	Many-to-One
block_requests	block_request_statuses	Many-to-One
otp_sessions	users	Many-to-One
operation_records	users (actor)	Many-to-One (nullable)
operation_records	roles	Many-to-One
operation_records	operation_types	Many-to-One
operation_records	operation_severities	Many-to-One

4.7 Key Design Decisions

UUID primary keys. All tables use UUID primary keys generated by the application layer (JPA `@GeneratedValue`). This avoids exposing predictable sequential IDs to clients and is compatible with distributed generation.

DECIMAL(19,2) for monetary values. The `balance` and `amount` columns use `DECIMAL(19,2)` (mapped to Java’s `BigDecimal`) rather than `FLOAT` or `DOUBLE`. Floating-point binary representation is inherently imprecise for decimal fractions — for example, $0.1 + 0.2$ in IEEE 754 double precision is `0.30000000000000004`, not `0.30`. For a banking system, this is unacceptable.

BTREE indexes. Three indexes are defined on `transactions` (`owner`, `account`, `createdAt`) and three on `operation_records` (`actorEmail`, `target`, `createdAt`). These cover the most frequent query patterns: loading a user’s transaction list and filtering the audit log by date.

accountName snapshot in transactions. The `accountName` column in `transactions` stores the account name at the time of the transaction. This is intentional denormalisation: if the account name changes later, the historical transaction record still shows the correct name.

actor nullable in operation_records. The `actor` FK in `operation_records` is nullable to allow system-generated events (e.g. the automatic seeding of demo data) to be recorded without a specific user actor. The `actorEmail` column stores a snapshot of the email regardless, so the record is always identifiable.

Chapter 5

Backend Architecture

5.1 Package Structure

The backend is a standard Spring Boot 3 application with the following package layout:

Backend package structure

```
com.example.banking
|-- controller/      REST controllers (HTTP layer)
|-- service/         Business logic
|-- model/           JPA entities (16 tables)
|-- repository/      Spring Data JPA repositories
|-- security/        JWT filter, JWT service, SecurityConfig
'-- dto/             Request and response objects
```

5.2 Security Filter Chain

Every HTTP request passes through the `JwtAuthenticationFilter` before reaching any controller. The filter performs the following steps:

1. Extract the **Authorization: Bearer <token>** header.
2. If no header is present, pass the request through (unauthenticated — Spring Security will enforce access rules downstream).
3. Parse and validate the JWT signature and expiry using `JwtService`.
4. Load the `User` entity from the database by the email claim.
5. Check that `user.accountStatus` is `ACTIVE` — a locked or blocked user is rejected even with a valid token.
6. Set a `UsernamePasswordAuthenticationToken` in the `SecurityContextHolder` with the user and their role.

Core JWT validation logic in `JwtAuthenticationFilter`

```

User user = userRepository.findByEmail(email).orElse(null);
if (user == null || !user.getAccountStatus().is(AccountStatus.ACTIVE)) {
    response.sendError(SC_UNAUTHORIZED, "User not found or access blocked");
    return;
}
UsernamePasswordAuthenticationToken auth =
    new UsernamePasswordAuthenticationToken(
        user, null,
        List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole().getCode()))
    );
SecurityContextHolder.getContext().setAuthentication(auth);
filterChain.doFilter(request, response);

```

5.3 JWT Implementation

Tokens are generated by `JwtService` using the `JJWT` library with HMAC-SHA256 signing. Each token contains:

- **Subject:** user email address.
- **role claim:** the user's role code (e.g. "CUSTOMER").
- **userId claim:** the user's UUID.
- **Issued at:** current instant.
- **Expiry:** issued-at plus one hour.

The signing secret is provided via the `APP_JWT_SECRET` environment variable and must be at least 32 characters long for HMAC-SHA256.

5.4 Two-Factor Authentication Flow

Login is a two-step process. A JWT is only issued after both steps succeed.

Step 1 — Password verification (POST `/api/auth/login`):

1. Look up the user by email.
2. Check `accountStatus` is `ACTIVE`.
3. Verify password against the BCrypt hash.
4. If wrong: increment `failedLoginAttempts`; if now ≥ 3 , set status to `LOCKED_LOGIN_FAILURE` and record a `CRITICAL` audit event.
5. If correct: reset `failedLoginAttempts` to 0; generate a 6-digit OTP; store its BCrypt hash in a new `OtpSession` (expires in 5 minutes, status `PENDING`); return the `otpSessionId`.

Step 2 — OTP verification (POST `/api/auth/verify-otp`):

1. Find `OtpSession` by ID where status = `PENDING`.
2. Check `expiresAt` has not passed.
3. Verify the submitted code against `codeHash` using BCrypt.
4. If wrong: increment `attempts`; if ≥ 3 , set status to `EXPIRED`.
5. If correct: set status to `USED`; update `user.lastLoginAt`; issue JWT.

5.5 Service Layer

Table 5.1: Core service classes and their responsibilities

Service	Responsibility
AuthService	Registration, login (2FA), OTP verification.
CustomerService	Account overview, transfers, payments, block requests, CSV downloads.
AdminService	Dashboard statistics, customer listing, security queue, account block/unblock, block request approval/rejection, user unlock.
OperationService	Records all audit events into <code>operation_records</code> .
BootstrapService	Seeds lookup tables (idempotently) and demo data on startup.

5.6 Exception Handling

`GlobalExceptionHandler` catches all exceptions thrown from services and maps them to appropriate HTTP status codes:

Table 5.2: Exception to HTTP status mapping

Exception	HTTP Status
<code>IllegalArgumentException</code>	400 Bad Request
<code>MethodArgumentNotValidException</code>	400 Bad Request
<code>EntityNotFoundException</code>	404 Not Found
<code>IllegalStateException</code>	423 Locked
<code>AccessDeniedException</code>	403 Forbidden
Exception (fallback)	500 Internal Server Error

All error responses follow a standard JSON envelope:

Standard error response format

```
{
  "timestamp": "2026-06-10T20:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Insufficient balance",
  "path": "/api/customer/transfers"
}
```

Chapter 6

Frontend Architecture

6.1 Overview

The frontend is a React 18 single-page application written in TypeScript, built with Vite, and using Ant Design for UI components. In production, the compiled static bundle is served by Nginx inside a Docker container.

6.2 Routing

React Router v6 is used for client-side routing. Routes are divided into three groups:

- **Public routes:** `/`, `/login`, `/register`, `/verify-otp`.
- **Customer routes:** `/customer/overview`, `/customer/accounts`, `/customer/payments`, `/customer/activity`.
- **Admin routes:** `/admin/overview`, `/admin/customers`, `/admin/operations`, `/admin/security`.

`ProtectedRoute` is a wrapper component that reads the JWT from `localStorage`, checks the user's role, and redirects to the login page if the user is unauthenticated or to their role's home page if they attempt to access a route outside their role.

6.3 Authentication Context

`AuthProvider` is a React context that wraps the entire application and exposes:

- `user` — the current `UserDto` loaded from `localStorage`.
- `token` — the JWT string stored in `localStorage`.
- `login()` — submits credentials, stores the `otpSessionId` in `sessionStorage`.
- `verifyOtp()` — submits the OTP, receives the JWT and stores it in `localStorage`.
- `logout()` — clears all storage and resets context state.

6.4 API Client

All HTTP requests go through a shared `api/client.ts` module that:

- Reads the JWT from `localStorage` automatically.
- Injects `Authorization: Bearer <token>` on every request.
- Sets `Content-Type: application/json` for request bodies.
- Throws a typed error if `response.ok` is false, including the HTTP status and server error message.

Chapter 7

API Design

7.1 Endpoint Overview

Table 7.1: REST API endpoints

Method	Path	Role	Description
POST	/api/auth/register	Public	Register new customer
POST	/api/auth/login	Public	Step 1: password check
POST	/api/auth/verify-otp	Public	Step 2: OTP check + JW
GET	/api/me	Any	Current user profile
GET	/api/health	Public	Health check
GET	/api/customer/overview	CUSTOMER	Dashboard data
GET	/api/customer/accounts	CUSTOMER	Account list
GET	/api/customer/activity	CUSTOMER	Transaction history
POST	/api/customer/transfers	CUSTOMER	Create transfer
POST	/api/customer/payments	CUSTOMER	Create payment
POST	/api/customer/accounts/{id}/request-block	CUSTOMER	Request block
GET	/api/customer/downloads/statement	CUSTOMER	CSV statement
GET	/api/customer/downloads/history	CUSTOMER	CSV history
GET	/api/admin/dashboard	ADMIN	System statistics
GET	/api/admin/customers	ADMIN	Customer list
GET	/api/admin/operations	ADMIN	Full audit log
GET	/api/admin/security	ADMIN	Security queue
POST	/api/admin/users/{id}/unlock-access	ADMIN	Unlock user
POST	/api/admin/accounts/{id}/block	ADMIN	Block account
POST	/api/admin/accounts/{id}/unblock	ADMIN	Unblock account
POST	/api/admin/block-requests/{id}/approve	ADMIN	Approve request
POST	/api/admin/block-requests/{id}/reject	ADMIN	Reject request

7.2 HTTP Status Codes

Table 7.2: HTTP status codes used by the API

Code	Meaning
200	Success
201	Resource created (registration)
400	Validation error, wrong password, insufficient balance
401	Missing or invalid JWT token
403	Insufficient role
404	Resource not found
423	Account or user is locked / blocked
500	Internal server error

Chapter 8

Security Model

8.1 Two-Factor Authentication

As described in Chapter 5, login requires both a correct password and a valid OTP. The OTP is a 6-digit numeric code that expires after five minutes. In this prototype, the OTP is written to the application log (prefixed with `[DEV]`). A production system would replace this with an email or SMS delivery.

The OTP code is never stored in plaintext — it is hashed with BCrypt before being written to the database. Verification uses `passwordEncoder.matches()`, the same mechanism used for password verification.

8.2 Account Locking

After three consecutive incorrect password entries, the user's `accountStatus` is set to `LOCKED_LOGIN_FAILURE`. The account is then rejected at two points:

- During login (before OTP generation).
- During every subsequent request (in `JwtAuthenticationFilter`, even with a valid token).

An administrator must explicitly call `POST /api/admin/users/{id}/unlock-access` to restore access. This resets `accountStatus` to `ACTIVE` and sets `failedLoginAttempts` to 0.

8.3 Account Blocking Workflow

Bank accounts (not user logins) can be blocked through two paths:

Customer-initiated:

1. Customer submits a block request with a reason.
2. Account status changes to `PENDING_BLOCK`.
3. Admin approves → account status becomes `BLOCKED`.
4. Admin rejects → account status reverts to `ACTIVE`.

Admin-initiated:

1. Admin calls the block endpoint directly.
2. Account status immediately becomes **BLOCKED**.
3. Any pending block request for that account is auto-approved.

Effect of blocking: Transfers and payments from a **BLOCKED** or **PENDING_BLOCK** account throw an exception before any balance mutation.

8.4 Audit Logging

Every security-relevant action produces a row in `operation_records` via `OperationService.record()`. Table 8.1 shows the severity levels.

Table 8.1: Audit log severity levels and example events

Severity	Example events
INFO	Login success, OTP verified, transfer created, history downloaded
WARNING	Failed login attempt, account block requested
CRITICAL	Account blocked after 3 login failures, account blocked by admin
SUCCESS	User access restored, account unblocked

8.5 Known Limitations

Table 8.2: Known security limitations

Area	Limitation
OTP delivery	OTP is only logged to console. No email/SMS delivery.
Race conditions	No optimistic locking (<code>@Version</code>) on balance updates. Two concurrent transfers from the same account could both pass the balance check.
Token revocation	Issued JWTs cannot be invalidated before expiry.
Rate limiting	No rate limiting on login or OTP endpoints.

Chapter 9

Testing

9.1 Test Strategy

The project uses three levels of testing:

- **Unit tests:** test individual service classes with all dependencies mocked using Mockito.
- **Integration tests:** start the full Spring Boot context with an H2 in-memory database and test HTTP endpoints via MockMvc.
- **End-to-end tests:** Playwright scripts run against the live Docker Compose stack and simulate real user interactions in a browser.

Run all backend tests with:

Running backend tests

```
docker run --rm \
  -v "$(pwd)/backend:/app" \
  -v "$HOME/.m2:/root/.m2" \
  -w /app maven:3.9.8-eclipse-temurin-21 \
  mvn test
```

9.2 Unit Tests

Table 9.1: Unit test classes and scenario counts

Class	Scenarios	What is verified
JwtServiceTest	5	Token generation, subject/role extraction, tamper detection, e
AuthServiceTest	9	Registration, login failures, locking, OTP expiry, wrong code,
CustomerServiceTest	8	Transfer validation, internal double-booking, payment, block r
AdminServiceTest	7	Unlock user, block/unblock account, approve/reject block req

9.3 Integration Tests

Table 9.2: Integration test classes

Class	Tests	What is verified
AuthIntegrationTest	5	Register 201, unknown email 404, wrong password 400, locked
CustomerIntegrationTest	3	401 without token, 403 for admin role
AdminIntegrationTest	3	401 without token, 403 for customer role
SecurityIntegrationTest	5	Health public, protected 401, cross-role 403

9.4 Test Results

All 46 tests pass on every build:

Maven test output

```
Tests run: 46, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
Total time: 29.845 s
```

9.5 Coverage Gaps

Table 9.3: Known test coverage gaps

Area	Gap
OTP delivery	No test for email/SMS delivery (console-only in prototype).
CSV downloads	<code>downloadStatement()</code> and <code>downloadHistory()</code> not directly tested.
Concurrent TX	No test for race condition on simultaneous transfers.
Frontend unit tests	No React component unit tests (Vitest / RTL).
Full E2E flow	Playwright suite covers happy paths only; no negative scenario automation.

Chapter 10

Application Walkthrough

This chapter provides a visual walkthrough of the application using screenshots taken from a live demo session. All screenshots are taken from the running Docker Compose stack.

10.1 Landing Page

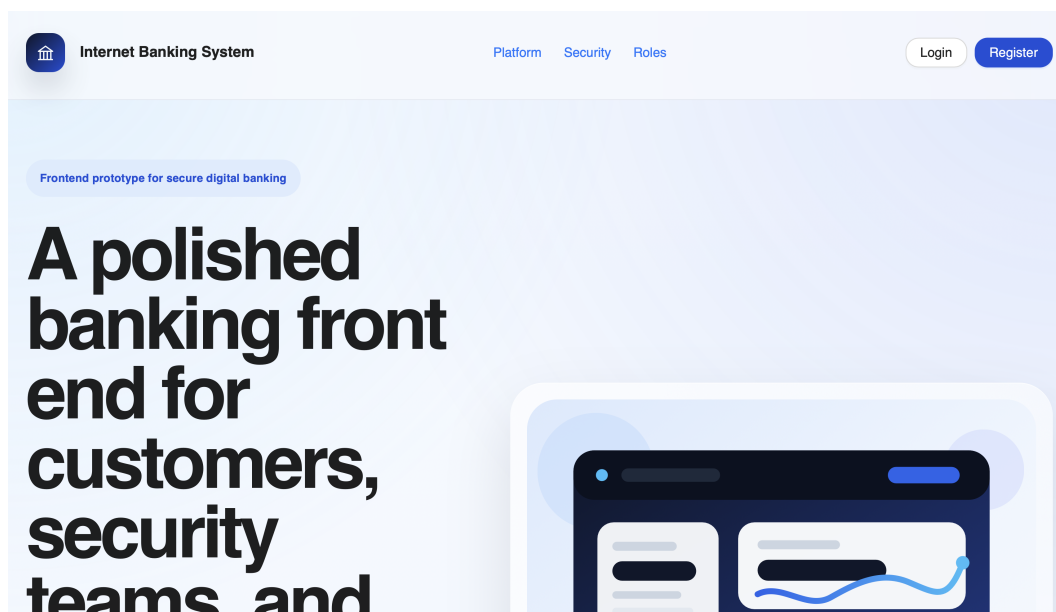


Figure 10.1: Landing page — hero section. The public homepage presents the platform concept and provides navigation to login and register.

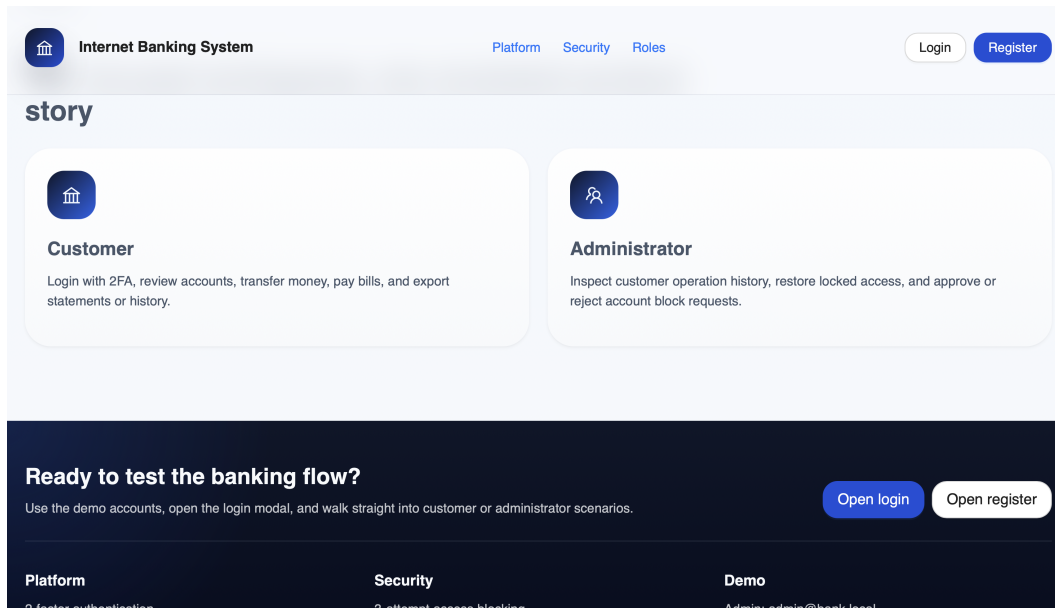


Figure 10.2: Landing page — roles section. Describes the two primary roles (Customer and Administrator) and offers quick access to the login modal with pre-filled demo credentials.

10.2 Login and Two-Factor Authentication

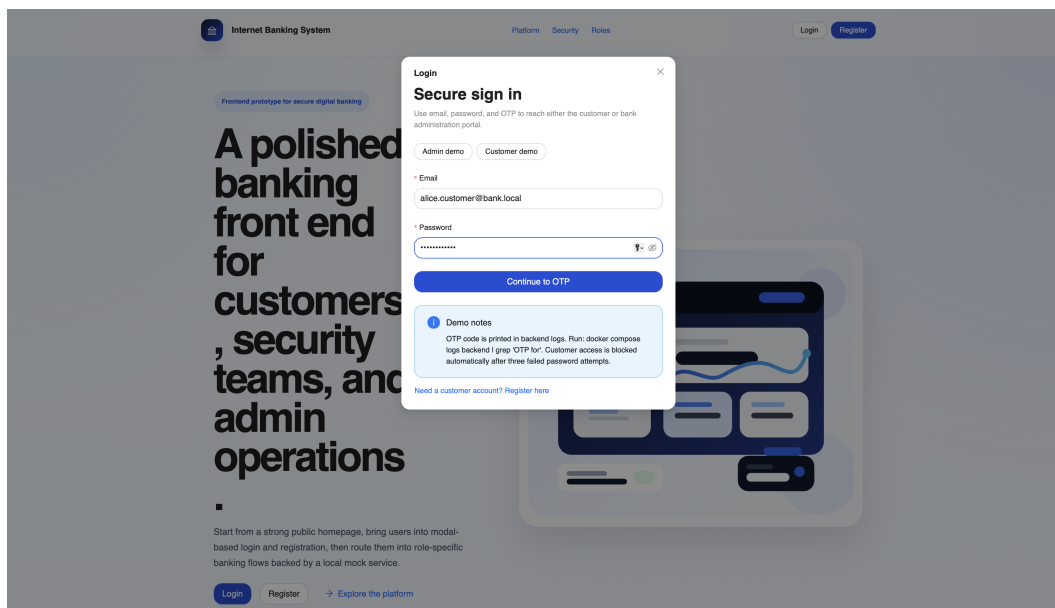


Figure 10.3: Login modal — step 1. The user enters their email and password. Demo credential shortcuts are provided for each role.

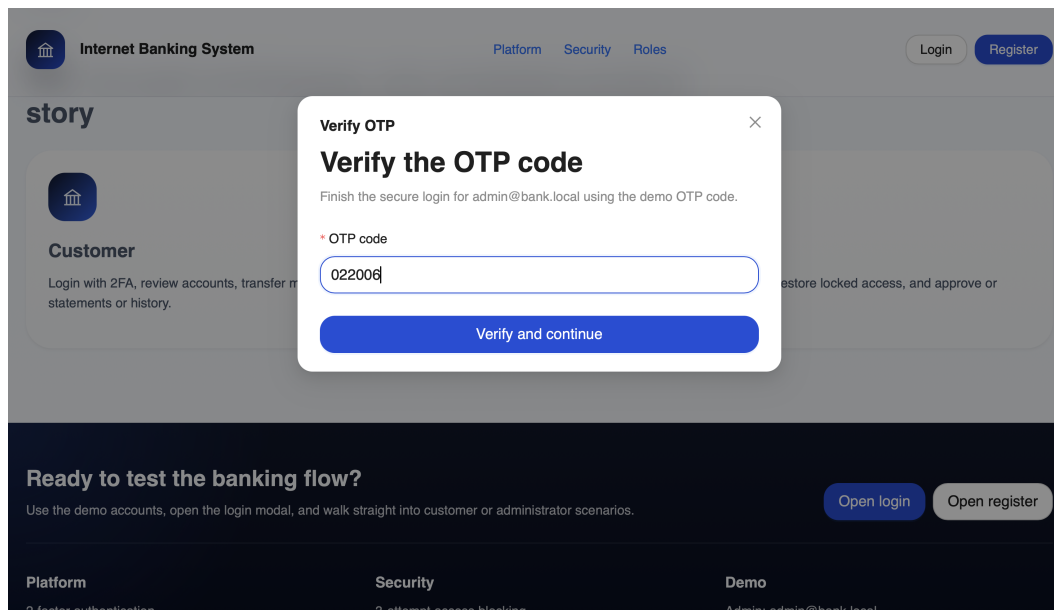


Figure 10.4: OTP verification modal — step 2. After a successful password check, the user must enter the 6-digit OTP code retrieved from the backend log ([DEV] OTP for admin@bank.local: 022006).

10.3 Admin Dashboard

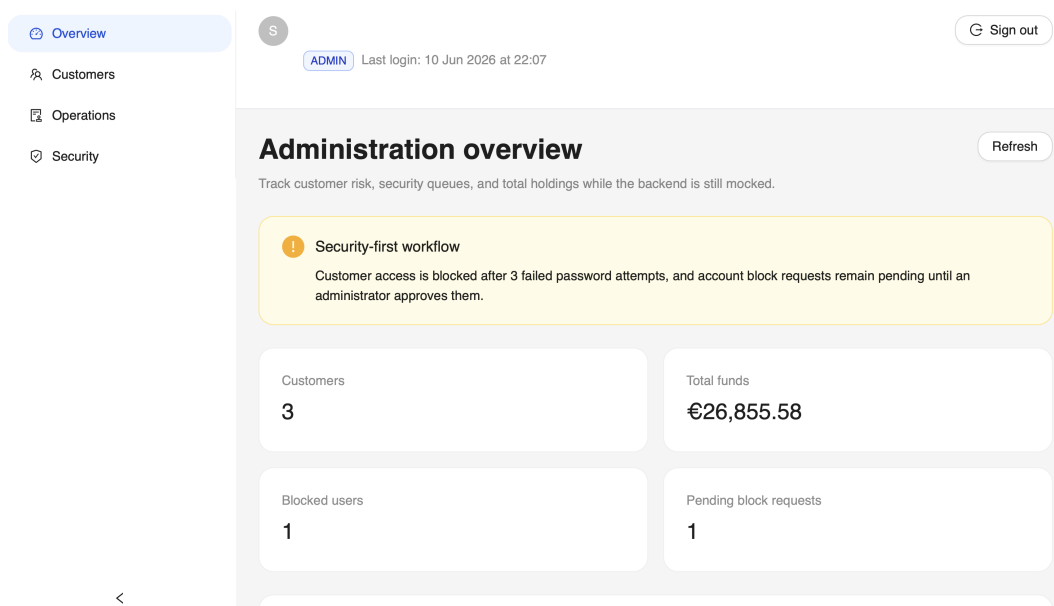


Figure 10.5: Admin overview — system statistics. Shows total customers (3), total funds across all accounts (26,855.58), blocked users (1), and pending block requests (1).

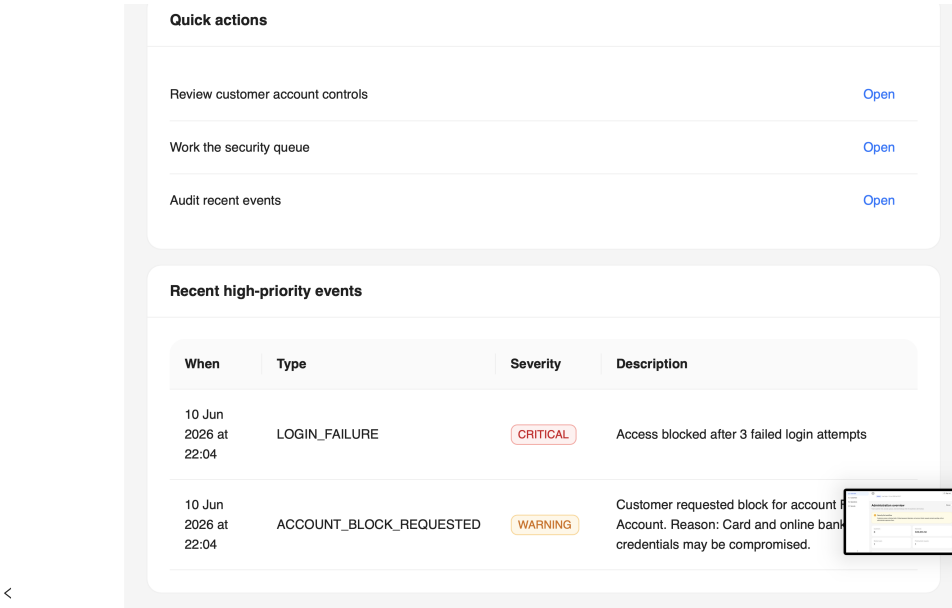


Figure 10.6: Admin overview — recent high-priority events. The dashboard shows the latest CRITICAL and WARNING events from the audit log, including the locked customer incident and Brian’s block request.

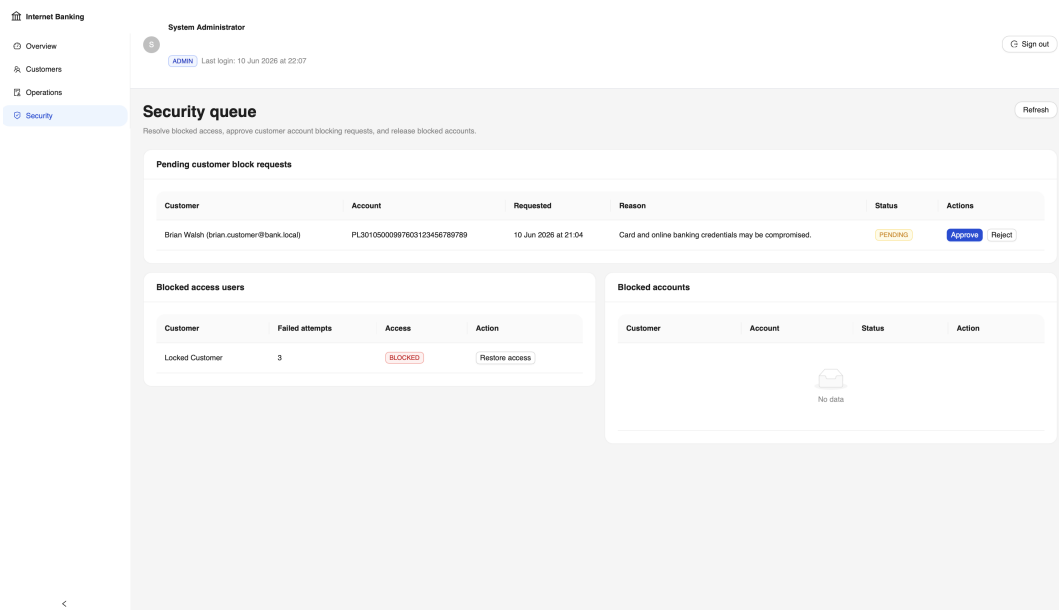


Figure 10.7: Admin security queue. Three sections: pending block requests (Brian Walsh’s request with Approve/Reject buttons), blocked access users (Locked Customer with Restore access), and blocked accounts (empty at this point).

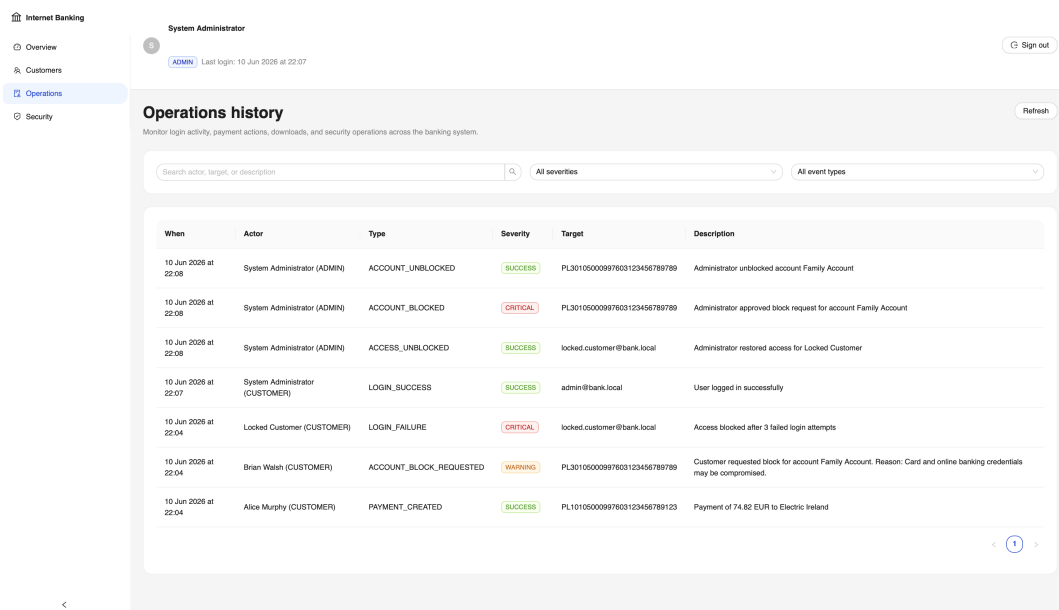


Figure 10.8: Admin operations — full audit log. Every action is recorded: account unblock, account block, access restore, login success, login failure, block request, and payment. Searchable and filterable by severity and type.

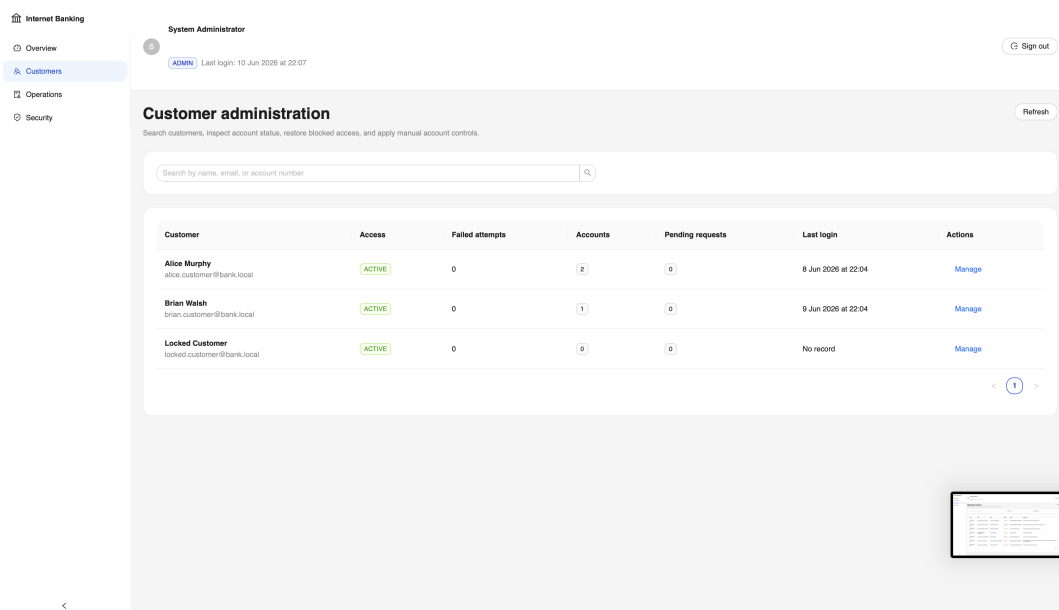


Figure 10.9: Admin customer administration. Lists all customers with access status, failed attempts, account counts, pending requests, and last login. A “Manage” action opens a customer detail drawer.

10.4 Customer Dashboard

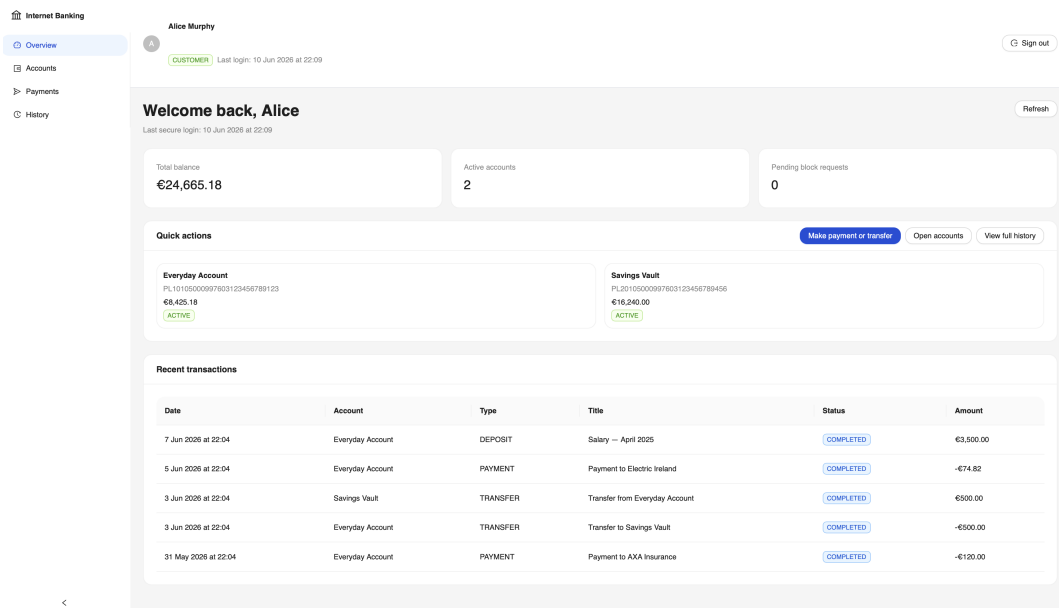


Figure 10.10: Customer overview — Alice Murphy’s dashboard. Shows total balance (24,665.18), 2 active accounts, and the 5 most recent transactions with types, statuses, and amounts.

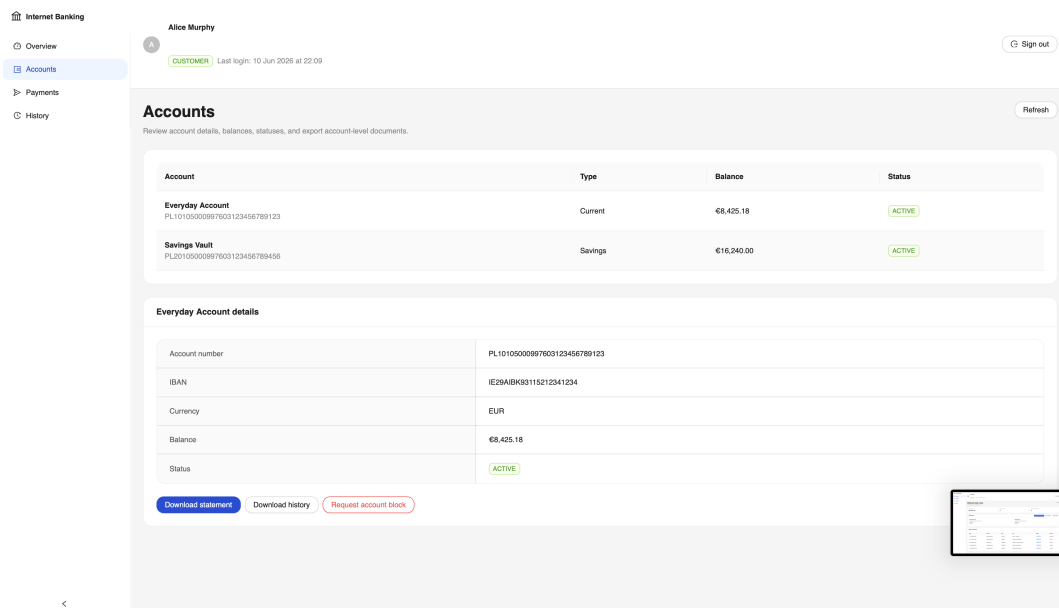


Figure 10.11: Customer accounts page. Shows both accounts with IBAN, account number, currency, balance, and status. The detail panel provides download and block request actions.

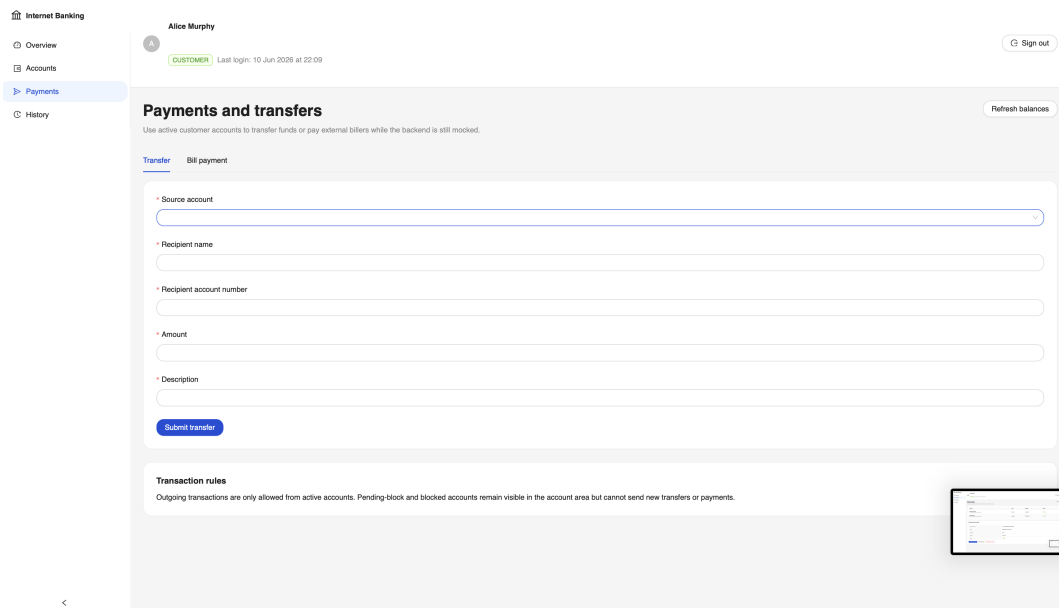


Figure 10.12: Payments and transfers page. The Transfer tab allows sending funds to an account number. The Bill Payment tab handles external payees. Only ACTIVE accounts are shown in the source account selector.

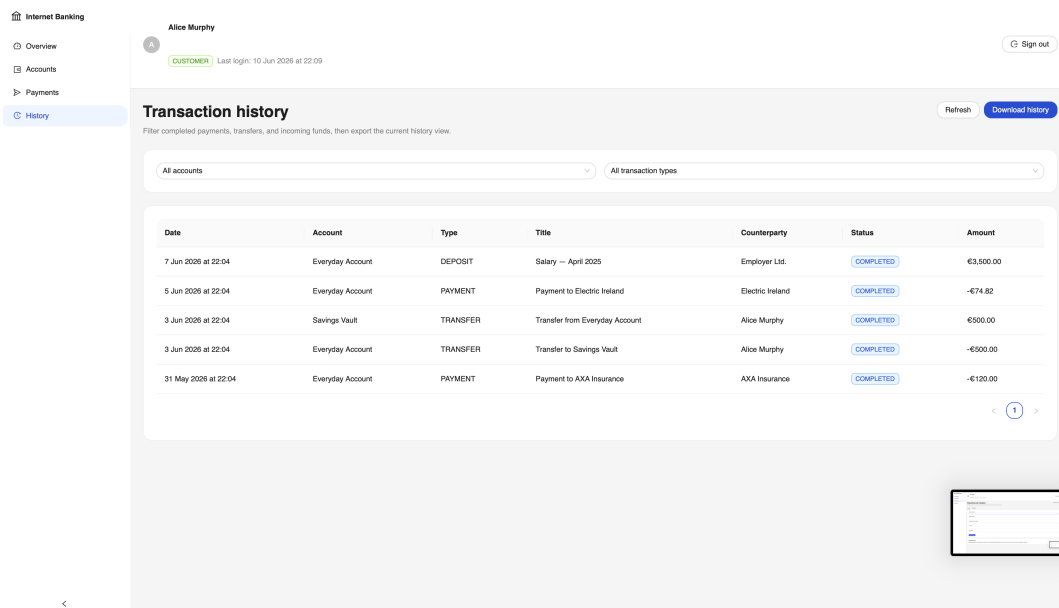


Figure 10.13: Transaction history page. Shows all transactions with date, account, type, title, counterparty, status, and signed amount. Filterable by account and transaction type, with a CSV download option.

Chapter 11

Deployment

11.1 Docker Compose Setup

The entire system is defined in a single `docker-compose.yml` at the repository root. Three services are configured:

Table 11.1: Docker Compose services

Service	Image	Port	Role
ib-db	postgres:16	5432	PostgreSQL database
ib-backend	Multi-stage Maven + JRE build	8080	Spring Boot API
ib-frontend	Multi-stage Node.js + Nginx build	5173	React SPA

11.2 Environment Variables

Table 11.2: Key environment variables

Variable	Default	Purpose
SPRING_DATASOURCE_URL	jdbc:postgresql://db:5432/internet_banking	DB connection
SPRING_DATASOURCE_USERNAME	bank_user	DB user
SPRING_DATASOURCE_PASSWORD	bank_pass	DB password
APP_JWT_SECRET	(change me)	JWT secret
APP_ADMIN_EMAIL	admin@bank.local	Admin email
APP_ADMIN_PASSWORD	Admin123!	Admin password
APP_SEED_DEMO_DATA	true	Seed demo data

11.3 Starting the Application

Full startup sequence

```
# Clone the repository
git clone <repo-url>
cd internet-banking-system

# Start all services (builds images if not cached)
docker compose up --build

# Access the application
# Frontend:  http://localhost:5173
# Backend:   http://localhost:8080
# Health:    http://localhost:8080/api/health

# Get OTP code during login
docker compose logs -f backend | grep "OTP for"

# Stop and remove all containers and volumes (clean reset)
docker compose down -v
```

Chapter 12

Conclusions

12.1 Summary

The Internet Banking System successfully demonstrates the design and implementation of a secure, role-based web banking application. The key outcomes of the project are:

- A fully functional three-tier web application deployable with a single command.
- A normalised relational database schema with 16 tables, lookup tables for all enumerated domains, and proper indexing.
- Two-factor authentication using BCrypt-hashed OTP codes.
- Role-based access control enforced at both the HTTP and business logic layers.
- Complete audit logging for all security-relevant actions.
- A comprehensive test suite with 46 passing tests across unit, integration, and end-to-end levels.

12.2 Lessons Learned

Enum-as-table pattern. The refactoring from plain VARCHAR enums to lookup tables significantly increased the complexity of the application code (each enum comparison now requires a database lookup or a helper method call), but the benefit is a self-documenting, database-enforced domain model. For a system where domain values rarely change, the VARCHAR approach is simpler; for a system that needs to support configuration of valid values at runtime, lookup tables are the correct choice.

Schema migration tooling. Using `ddl-auto: update` in Hibernate is convenient for development but caused issues when switching from the old schema (with VARCHAR enums) to the new schema (with FK columns). In a production system, a proper migration tool (Flyway or Liquibase) would manage schema changes safely without requiring a full database wipe.

BigDecimal for money. The use of `DECIMAL(19,2)` and Java's `BigDecimal` for all monetary values was essential. Float/double would introduce rounding errors in financial calculations.

12.3 Potential Improvements

- **OTP delivery:** integrate a real email or SMS provider.
- **Optimistic locking:** add `@Version` to `BankAccount` to prevent race conditions on concurrent transfers.
- **Token revocation:** implement a token blacklist or use short-lived refresh tokens to allow logout to be enforced server-side.
- **Rate limiting:** add rate limiting on authentication endpoints to slow brute-force attacks beyond the 3-strike lock.
- **Schema migrations:** replace `ddl-auto: update` with Flyway for controlled, versioned schema changes.
- **Pagination:** add cursor-based pagination to transaction history and audit log endpoints for large datasets.

References

1. Spring Boot Documentation. *Spring Boot Reference Guide 3.3*. Pivotal Software, 2024. <https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/>
2. Spring Security Documentation. *Spring Security Reference 6.x*. Pivotal Software, 2024. <https://docs.spring.io/spring-security/reference/>
3. JJWT Library. *Java JWT: JSON Web Token for Java and Android*. Stormpath / jwtk, 2024. <https://github.com/jwtk/jjwt>
4. React Documentation. *React 18 — The Library for Web and Native User Interfaces*. Meta, 2024. <https://react.dev/>
5. Ant Design. *Ant Design 5.x Component Library*. Ant Group, 2024. <https://ant.design/>
6. PostgreSQL Documentation. *PostgreSQL 16 Documentation*. The PostgreSQL Global Development Group, 2024. <https://www.postgresql.org/docs/16/>
7. Hibernate ORM. *Hibernate ORM 6.5 User Guide*. Red Hat, 2024. https://docs.jboss.org/hibernate/orm/6.5/userguide/html_single/
8. Docker Documentation. *Docker Compose Reference*. Docker Inc., 2024. <https://docs.docker.com/compose/>
9. Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
10. OWASP. *OWASP Top Ten — Web Application Security Risks*. OWASP Foundation, 2023. <https://owasp.org/www-project-top-ten/>